Assignment 3 KBAI

Rimski Logtens, S1138380
Sem de Vries, S1093742

19 December 2025

Contents

1 Introduction

"This report discusses the 'Sudoku' assignment, which focuses on the AC-3 algorithm and the implementation of backtracking. The assignment was structured so that the base code was provided, while specific core functions required implementation.

The provided functions handled reading the text files containing the Sudoku puzzles, printing the output, and parsing the data into variables. Consequently, the work focused on implementing the AC-3 algorithm, a function to identify neighboring cells (constraints), and a function to check the validity of the final solutions."

1.1 Background of Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem (CSP) refers to a mathematical framework used to model problems where the objective is to find a state that meets a comprehensive set of requirements. With this approach, the main focus is on defining the problem state rather than the specific procedures to solve it (Russell & Norvig, 2021). A CSPs is a problem which is builds upon variables, values, and a set of constraints. These constraints make sure that restrictions are applied. These constraints are formalized as logical rules which ensure that only the valid solution is allowed .
This approach is very useful for scheduling, map coloring, and puzzles like Sudoku where the relationships are fixed but the arrangement is complex. (Brailsford et al., 1999) In such contexts, CSPs allow for a general-purpose solving engine to be applied to a wide variety of domains efficiently, providing a structured method for handling complex logical dependencies.

2 Methodology

2.1 Problem representation CSP's

For this project the Constraint satisfaction problem which needed to be solved was a Sudoku. The way a Sudoku is constructed consists of multiple nine different 9x9 value cells which need to be solved with constraints. So if the sudoku problem is put into terms of a CSP there is a set of variables $X = \{x_1, x_2, x_3, x_n\}$ which are the 9x9 value cells, then there is the domain which in the case of a sudoku represent the numbers which need to be put within the cell $D = \{D_1, D_2, D_3, D_n\}$ and then there are the set of constraints which are that every number from the domain can only be used one time vertically, one time horizontally, one time within every cell of the 9x9. This therefore proposes the CSP algorithms!

2.2 Representing the AC-3 Algorithm

The arc consistency algorithm number 3 (AC-3) is a constraint propagation algorithm which is used to solve Constraint Satisfaction problems. Its primary goal is to enforce arc consistency across the constraint graph. Within a ac-3 algorithm **Initialization** begins by populating a processing queue with all binary constraints (arcs) existing between every cell and its neighbors in the 9×9 grid. The core **Processing Loop** iteratively selects an arc (X_i, X_j)—potentially prioritized by heuristics like Minimum Remaining Values—and validates it via the REVISED function. This function performs the **Revision Step**, checking if every value in the domain of X_i has compatible support in X_j; if a conflict is found, the unsupported value is pruned to eliminate impossible solutions. Crucially, this triggers a **Propagation Phase**: if a domain is modified, all other neighbors of X_i are re-added to the queue, ensuring that the new constraint information ripples through the graph until stability is achieved or a domain wipe-out detects failure.

2.3 Heuristics

For the report we used 3 different heuristics which could be used to solve the provided sudoku's. These heuristics are: first in first out (fifo), minimum Remaining values (mrv) and constrain. The fifo algorithm is the baseline approach where the AC-3 algorithm treats the collection of arcs as a queue. Where they are processed in a first in and first out manner, this therefore does not prioritize any specific arc over another arc. The Mimimum remaining values heuristic prioritizes the variables which have the fewest possibilities first, therefore the algorithm can detect failures as quickly as possible which makes the pruning of the search tree more efficient. (Haralick & Elliot, 1980) For the constraint heuristic the cells which can only have one possible value act as a rigid constraint to its neighbours. Therefore these arcs are processed first making sure that the definite values are propagated immediately to neighboring cells therefore reducing

the domains more quickly.

2.4 Pseudocode Get Neighbour function

Neighbor Identification Logic The initialization of the Sudoku problem requires transforming the static grid into a connected constraint graph. The AddNeighbours function iterates through every cell in the 9×9 grid to establish these connections. For any given cell at coordinates (row,col), the algorithm identifies "peers"—cells that cannot hold the same value due to Sudoku rules. These peers are located in three specific units: the same row, the same column, and the encompassing 3×3 subgrid. To ensure efficiency and avoid redundant edges, the algorithm collects these coordinates into a set before storing them as a list of neighbor references within the Field object. This pre-calculation step is crucial, as it allows the subsequent AC-3 algorithm to retrieve constraints in O(1) time during propagation.

Algorithm 1 Add Neighbors to Fields

```
function AddNeighbours(grid)
    size ← 9
    for row ← 0 to size − 1 do
        for col ← 0 to size − 1 do
            neighbors ← ∅
                                                    ▷ Add Row Peers
            for j ← 0 to size − 1 do
                if j ≠ col then neighbors ← neighbors ∪ {(row, j)}
                end if
            end for
                                                    ▷ Add Column Peers
            for i ← 0 to size − 1 do
                if i ≠ row then neighbors ← neighbors ∪ {(i, col)}
                end if
            end for
                                                    ▷ Add 3 × 3 Box Peers
            box_row ← (row//3) × 3
            box_col ← (col//3) × 3
            for i ← box_row to box_row + 2 do
                for j ← box_col to box_col + 2 do
                    if i ≠ row ∨ j ≠ col then
                        neighbors ← neighbors ∪ {(i, j)}
                    end if
                end for
            end for

            grid[row][col].SetNeighbours(neighbors)
        end for
    end for
end function
```

2.5 Pseudocode Solve Function

AC-3 with Heuristic Queue Management The core solving mechanism is an implementation of the AC-3 algorithm, designed to enforce arc consistency across the board. The procedure begins by initializing a queue with all binary constraints (arcs) present in the puzzle. A distinct feature of this implementation is the variable selection strategy for processing the queue. While a standard AC-3 algorithm processes the queue in a First-In-First-Out (FIFO) manner, this implementation supports dynamic heuristics to optimize convergence: FIFO (Baseline): Processes arcs in the order they were added. MRV (Minimum Remaining Values): Prioritizes arcs where the variable has the smallest current domain size, effectively targeting the most constrained cells first to detect failures early. Constrain: Prioritizes arcs connected to fields that are already "finalized" (domain size of 1), propagating the strongest constraints immediately. If the Revised function modifies a domain, the algorithm checks for domain wipe-out (an empty domain indicating failure). If the domain remains valid but changed, all neighbors of the current cell are re-added to the queue to ensure the change is propagated throughout the constraint graph.

Algorithm 2 AC-3 Solver with Heuristics

```
function SOLVE(sudoku, heuristic)
    board ← sudoku.GETBOARD
    queue ← all arcs (field, neighbor) in board

    while queue is not empty do
                                          ▷ Select arc based on heuristic
        if heuristic = 'mrv' then
            (curr, neighbor) ← arc with min domain size in queue
        else if heuristic = 'constrain' then
            (curr, neighbor) ← arc where domain size is 1
        else                                                ▷ Default FIFO
            (curr, neighbor) ← first item in queue
        end if
        Remove (curr, neighbor) from queue

        if REVISED(curr, neighbor) then
            if curr.GETDOMAINSIZE = 0 then
                return False                          ▷ Solution impossible
            end if

            for each other in curr.GETOTHERNEIGHBOURS(neighbor) do
                Add (other, curr) to queue
            end for
        end if
    end while
    return True
end function
```

2.6 Pseudocode Valid solution

Solution Verification To ensure the robustness of the solver, the ValidSolution function performs a two-phase check on the final board state. The first phase validates the assigned values (finalized cells). It iterates through every cell with a fixed value and ensures that no neighboring cell holds the exact same value, thereby satisfying the fundamental "all-different" constraint of Sudoku.
The second phase verifies the consistency of the remaining domains for non-finalized cells. It checks "arc consistency" by ensuring that for every value remaining in a cell's domain, there exists at least one compatible value in the domain of each neighbor. If a value lacks support from a neighbor, it indicates that the constraint propagation was incomplete or that the solution state is invalid.

Algorithm 3 Check Solution Validity

```
1:  function ValidSolution(sudoku)
2:      board ← sudoku.GetBoard
3:                                          ▷ Phase 1: Check Fixed Value Conflicts
4:      for each field in board do
5:          value ← field.GetValue
6:          if value ≠ 0 then
7:              for each neighbor in field.GetNeighbours do
8:                  if neighbor.GetValue = value then
9:                      return False                          ▷ Conflict found
10:                 end if
11:             end for
12:         end if
13:     end for
14:                                  ▷ Phase 2: Check Arc Consistency of Domains
15:     for each field in board do
16:         for each neighbor in field.GetNeighbours do
17:             for each val in field.GetDomain do
18:                 support ← False
19:                 for each n_val in neighbor.GetDomain do
20:                     if val ≠ n_val then
21:                         support ← True
22:                         break
23:                     end if
24:                 end for
25:                 if not support then
26:                     return False                  ▷ Arc Inconsistency found
27:                 end if
28:             end for
29:         end for
30:     end for
31:     return True
32: end function
```

3 Results

To evaluate the impact of the heuristics, the algorithm was executed under three configurations: First in First out (FIFO), Minimum Remaining Values, and Least Constraining Value (LCV). For each configuration, the amount of revise calls, total domain removals and the max queue-size are counted and saved in a variable. The revise calls measure how many times `def revised(self current_field neighbour_field` was called. Domain removals counts how many times a a domain was deleted from an index in current domain. The max queue size measures the maximum numbers or arcs waiting to be processed at any point in the algorithm. These variables are chosen so that all heuristics perform an identical amount of fundamental work (the same number of revise calls and domain removals), ensuring a fair comparison. What the heuristics are expected to influence is the ordering of the arc process by selecting more informative arcs earlier, which should reduce the maximum queue size. As a result, the algorithm should converge faster despite performing the same underlying logical operations. The FIFO heuristics is chosen as a base case algorithm, as it does not contain a special heuristic and should therefore contain a larger maximum queue size than the MRV and LCV heuristics.

All variables are counted as numbers, as max queue-size is measured as `len(queue_size)`, and `revise calls` and `domain removals` as simple counters in the algorithm.

Metric	Revise fieldss	Total domain removals	Max queue size
Sudoku 1	9220	400	5782
Sudoku 2	9372	408	6025
Sudoku 3	8460	360	6348
Sudoku 4	6731	269	5141
Sudoku 5	8612	368	5398

Table 1: FIFO Metrics

Metric	Revise fields	Total domain removals	Max queue size
Sudoku 1	9220	400	1620
Sudoku 2	9372	408	1620
Sudoku 3	8460	360	1620
Sudoku 4	6731	269	1620
Sudoku 5	8612	368	1620

Table 2: MRV Metrics

Metric	Revise fields	Total domain removals	Max queue size
Sudoku 1	9220	400	7608
Sudoku 2	9372	408	7802
Sudoku 3	8460	360	7225
Sudoku 4	6731	269	5666
Sudoku 5	8612	368	7047

Table 3: LCV Metrics

3.1 Runtime complexity

We first apply the AC-3 algorithm to reduce the search space before backtracking. AC-3 creates arc-consistencies by repeatedly checking arcs, and removing values from a fields domain if they cannot exist within the domain of a neighboring field. Each arc-consistency check runs in $O(d^2)$ time, where d is the domain size of a field. Because every arc may need to be reconsidered up to d times as domains shrink, the total AC-3 cost is $O(Ed^3)$, where E is the number of arcs in the constraint graph. The use of python lists for queue operations adds an additional $O(E^2d)$ time, due to repeated linear scans and middle removals under different heuristics. For the fixed 9x9 Sudoku's this entire process runs in effectively constant time, but the cost increases with both the size of the constraint graph and domain size.

After AC-3, the algorithm performs a recursive backtracking search. Each search node performs domain finalization, conflict checks, and full board copying and restoration, giving a per-node cost of $O(Nd^{U+2})$, where U is the number of unresolved variables and N is the total number of fields.

This leaves a final big$O$ complexity of: $O(Ed^3 + E^2d + Nd^{U+2})$.

4 Discussion

Within the results the most important difference noted was that the max queue size of MRV was siginificantly different from FIFO and LCV. This difference in value was very obvious due to the max queue size of MRV being flat out 1620 whilst the queue size of fifo was ranging between 5782 and 6348, whilst LCV had a queue size ranging between 5666 and 7802. This can be clarified by the way that the fifo algorithm is less efficient in processing due to the FIFO heuristics using the principle that the first arc which gets in is the first arc to go out. Therefore this does not prioritize the most optimal arc queue. As for why the LCV performed worse is because the LCV prioritizes finalized cells which have a domain of only 1 meaning that it doesn't propogate as efficient therefore creating a larger queue size. Therefore we can conclude that MRV is the most efficient heuristic in our test case with 5 Sudoku's. An observation was made with the MRV heuristic that for all the 5 different Sudoku's the same queue size was yielded which was not the case for the other heuristics!

5 Conclusion

This project implemented the AC-3 algorithm for solving Constraint Satisfaction Problems (CSPs). We utilized this algorithm with three different arc selection strategies: a baseline measure (FIFO), the Minimum Remaining Values (MRV) heuristic, and the Least Constraining Value (LCV) heuristic. In conclusion, the Minimum Remaining Values (MRV) heuristic proved to be the most effective strategy. This finding is strongly supported by the results section, specifically by the fact that the MRV implementation generated the smallest maximum queue size. This reduced maximum queue size indicates that the MRV strategy successfully focused constraint propagation on the most critical (most constrained) cells. This efficient targeting led to significantly fewer total constraint re-propagations across the entire graph, ultimately allowing the Sudoku CSP problem to be solved with greater operational efficiency.

Collaboration within the group functioned effectively, with responsibilities for implementation, experimentation, and reporting shared evenly between Sem and Rimski

References

Brailsford, S. C., Potts, C. N., & Smith, B. M. (1999). Constraint satisfaction problems: Algorithms and applications [Publisher: Elsevier]. *European journal of operational research*, *119*(3), 557–581.

Haralick, R. M., & Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems [Publisher: Elsevier]. *Artificial intelligence*, *14*(3), 263–313.

Norvig, P., & Russell, S. J. (1995). Artificial intelligence a modern approach.